Backpropagating Through Simulation: Analytic Policy Gradients for Sample and Learning Efficient Differentiable Continuous Control

Yueci Deng

Abstract

Model-free reinforcement learning algorithms such as Proximal Policy Optimization (PPO) treat the environment as a black box, estimating policy gradients from sampled rewardsa process that demands millions of interactions and relies on high-variance advantage estimates. When environment dynamics are differentiable, the return is an end-to-end differentiable function of the policy parameters, enabling exact gradient computation via backpropagation through simulation. We term this approach Analytic Policy Gradients (APG) and evaluate it against PPO on four continuous control tasks of increasing dynamical complexity: a one-dimensional point-mass target-reaching task, a 2D point-mass navigation task with obstacle avoidance, a 2D rigid-body T-block pushing task, and a 7-DOF Franka FR3 end-effector reaching task. Both algorithms share identical model architectures, observation normalization, and optimizer settings. To decouple sample efficiency from compute efficiency, we design a multi-axis evaluation protocol that records performance against environment steps and gradient steps. We report a segmented backpropagation scheme with MC and critic-based bootstrap modes that mitigates gradient degradation on long-horizon tasks, and present ablations over segment length and bootstrap strategy.

1 Introduction

1.1 Motivation

Reinforcement learning (RL) has achieved remarkable success in domains such as game playing, robotics, and autonomous control. However, poor sample efficiency remains a core bottleneck that prevents its deployment in the real world. In most practical tasks, every environment interaction incurs non-negligible costs in terms of time, money, or safety. Sample efficiency also remains a fundamental challenge: model-free algorithms such as PPO [1] require millions of environment interactions because they treat dynamics as a black box, estimating policy gradients from observed reward signals via Monte Carlo returns or learned value functions, which inevitably introduces high variance and estimation bias

A largely underexploited opportunity is that many modern simulators are inherently differentiable. Physics engines, rendering pipelines, and procedural task generators routinely expose analytic derivatives. When these are available, the expected return becomes a differentiable function of the policy parameters, and exact gradients can be obtained by backpropagating through the dynamicseliminating the need for advantage estimation [2], importance sampling, or entropy regularization.

1.2 Paper Organization

The remainder of this report is organized as follows. Section 2 reviews related work. Section 3 covers preliminaries on policy gradient methods and PPO. Section 4 describes the APG

methodology in detail. Section 5 presents the four benchmark environments. Section 6 details the experimental setup and provides result templates. Section 7 discusses findings and concludes.

2 Related Work

2.1 Policy Gradient Methods

Policy gradient methods optimize parameterized policies by estimating gradients of the expected return. REINFORCE [15] introduced the score-function estimator, which trades bias for variance and requires large sample counts. PPO [1] improves stability via a clipped surrogate objective, while GAE [2] reduces advantage estimation variance through a λ-weighted multi-step return. Despite these advances, all score-function methods treat environment dynamics as a black box and cannot exploit analytic structure in the transition function.

2.2 Differentiable Simulation

A parallel line of work develops simulators that expose analytic derivatives, enabling gradient-based policy optimization. Degrave et al. [9] demonstrated end-to-end gradient flow through rigid-body physics. Suh et al. [11] provided a systematic analysis of when differentiable simulators yield better policy gradients than score-function estimators, identifying the smoothness of the reward landscape and the magnitude of dynamics Jacobians as key determinants. Zhong et al. [10] applied differentiable simulation to system identification, showing that analytic gradients substantially accelerate parameter inference.

2.3 Short-Horizon and Segmented Backpropagation

Backpropagating through long trajectories amplifies vanishing and exploding gradient problems due to compounding Jacobians [8]. Xu et al. [5] addressed this by truncating gradient horizons and combining analytic rollouts with a learned value baseline, demonstrating that short-horizon analytic gradients can be effectively parallelized across many simulation workers for faster wall-clock convergence. Guo et al. [4] introduced Short-Horizon Actor-Critic (SHAC), which explicitly decouples the horizon over which analytic gradients are computed from the full episode horizon, using a separately trained critic to bootstrap returns beyond the truncation boundary. Our segmented backpropagation scheme (Section 4.3) follows this design, and we additionally compare MC and critic bootstrap strategies with a systematic segment-length ablation.

2.4 Model-Based RL and World Model Gradients

Model-based RL methods learn a world model and optimize policies through imagined rollouts. DreamerV3 [6] trains a recurrent latent world model and backpropagates policy gradients through multi-step imagination, achieving strong performance on diverse continuous control tasks without environment-specific differentiability. TD-MPC2 [7] combines a latent world model with model predictive control and analytic gradient updates, demonstrating scalability across robot manipulation and locomotion benchmarks. In contrast to these methods, our APG framework operates directly on ground-truth differentiable simulation dynamics, eliminating model bias but requiring an explicitly differentiable environment.

2.5 Natural Gradient and Second-Order Methods

Trust Region Policy Optimization (TRPO) [16] and natural policy gradient methods [13, 14] constrain policy updates via the Fisher information matrix to improve optimization geometry.

These methods also treat the environment as a black box but use second-order curvature information to take larger, safer steps. Our focus differs: rather than improving the optimizer, APG eliminates the variance introduced by score-function gradient estimation by exploiting environment differentiability.

3 Preliminaries

We begin this section by formulating the RL problem and then revisiting the PPO algorithm, which serves as a key baseline in prior work.

3.1 Markov Decision Processes

We model the RL problem as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the stochastic transition kernel, $R(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. A policy $\pi_{\boldsymbol{\theta}}(a \mid s)$ parameterized by $\boldsymbol{\theta}$ maps states to action distributions. The objective is to maximize the infinite-horizon discounted return:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \tag{1}$$

In practice, training and evaluation use a finite horizon H. By the value truncation lemma, the infinite sum can be approximated by the truncated episode return:

$$J_H(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}} \left[\sum_{t=0}^{H} \gamma^t R(s_t, a_t) \right]. \tag{2}$$

3.2 Proximal Policy Optimization (PPO)

PPO [1] is a model-free, on-policy algorithm that optimizes a clipped surrogate objective:

$$\mathcal{L}^{\text{PPO}}(\boldsymbol{\theta}) = \mathbb{E}_t \left[\min \left(r_t(\boldsymbol{\theta}) \hat{A}_t,\ \text{clip}\left(r_t(\boldsymbol{\theta}), 1-\epsilon, 1+\epsilon\right) \hat{A}_t \right) \right] \tag{3}$$

where $r_t(\boldsymbol{\theta}) = \frac{\pi_{\boldsymbol{\theta}}(a_t|s_t)}{\pi_{\boldsymbol{\theta}_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the advantage estimate computed via Generalized Advantage Estimation (GAE) [2], and ϵ is the clipping coefficient.

PPO treats the environment as a **black box**: it collects trajectories using the current policy, estimates advantages from observed rewards, and updates the policy without any knowledge of environment dynamics. The total PPO loss combines the clipped policy objective, a value function loss, and an entropy bonus:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}^{\text{PPO}}(\boldsymbol{\theta}) - c_1 \mathcal{L}^{\text{VF}}(\boldsymbol{\theta}) + c_2 H(\pi_{\boldsymbol{\theta}}) \tag{4}$$

4 Methodology

When the transition dynamics and reward function are both differentiable and known, the expected discounted return $J(\boldsymbol{\theta})$ becomes an endtoend differentiable function of the policy parameters $\boldsymbol{\theta}$. This makes it possible to treat the environment as a differentiable computational graph and apply Analytic Policy Gradients (APG) to learn the policy.

4.1 Analytic Policy Gradients

In contrast to black-box methods, APG assumes that the general stochastic transition kernel $P(s' \mid s, a)$ is replaced by a deterministic differentiable transition map $T(s, a)$. Together with a differentiable reward R_θ, this makes the rollout itself a differentiable computation graph. For a stochastic policy with reparameterized sampling $a_t = \mu_\theta(s_t) + \sigma_\theta \odot \epsilon_t$, $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and deterministic transition $s_{t+1} = T(s_t, a_t)$, the finite-horizon return is directly differentiable with respect to θ:

$$J(\theta) = \sum_{t=0}^{H} \gamma^t R_\theta(s_t, a_t), \quad s_{t+1} = T(s_t, a_t), \quad a_t = \mu_\theta(s_t) + \sigma_\theta \odot \epsilon_t. \tag{5}$$

The gradient $\nabla_\theta J(\theta)$ is obtained by applying the *total* derivative through the entire trajectory. Because each state s_t is a function of θ through all prior actions, we must use total derivatives $\frac{d}{d\theta}$ rather than partial derivatives:

$$\frac{dJ}{d\theta} = \sum_{t=0}^{H} \gamma^t \frac{dR_\theta(s_t, a_t)}{d\theta} \tag{6}$$

Applying the chain rule to each reward term, and accounting for the dependence of both s_t and a_t on θ:

$$\frac{dR_\theta(s_t, a_t)}{d\theta} = \frac{\partial R_\theta}{\partial a_t}\frac{da_t}{d\theta} + \frac{\partial R_\theta}{\partial s_t}\frac{ds_t}{d\theta} \tag{7}$$

The action total derivative follows from the reparameterization $a_t = \mu_\theta(s_t) + \sigma_\theta \odot \epsilon_t$:

$$\frac{da_t}{d\theta} = \frac{\partial \mu_\theta(s_t)}{\partial \theta} + \frac{\partial \mu_\theta(s_t)}{\partial s_t}\frac{ds_t}{d\theta} + \frac{d\sigma_\theta}{d\theta} \odot \epsilon_t \tag{8}$$

The state total derivative satisfies the recursion obtained by differentiating $s_t = T(s_{t-1}, a_{t-1})$:

$$\frac{ds_t}{d\theta} = \frac{\partial T}{\partial a_{t-1}}\frac{da_{t-1}}{d\theta} + \frac{\partial T}{\partial s_{t-1}}\frac{ds_{t-1}}{d\theta}, \qquad \frac{ds_0}{d\theta} = \mathbf{0} \tag{9}$$

Equation (9) is precisely backpropagation through time (BPTT) [3]: gradients flow backward through the dynamics Jacobians $\frac{\partial T}{\partial s_t}$ and $\frac{\partial T}{\partial a_t}$, accumulating contributions from all future reward terms. Equations (8) and (9) are mutually recursive: $\frac{da_t}{d\theta}$ depends on $\frac{ds_t}{d\theta}$, which in turn depends on $\frac{da_{t-1}}{d\theta}$. Because actions are sampled via the reparameterization trick, $\frac{da_t}{d\theta}$ carries gradients through both $\boldsymbol{\mu_\theta}$ and $\boldsymbol{\sigma_\theta}$. The entire recursion is unrolled automatically by any reverse-mode autodiff framework (e.g. PyTorch's `loss.backward()`), requiring no advantage estimation, importance sampling, or variance reduction.

The trade-off is that APG requires the transition dynamics T and the reward function R to be differentiable, which restricts the class of applicable environments and models. Furthermore, gradient chains that span the full horizon H can suffer from vanishing or exploding magnitudes, making the optimization unstable and slow to converge. This directly motivates the segmented backpropagation scheme presented in Section 4.3, which truncates the gradient flow into shorter segments to improve numerical stability and learning efficiency.

4.2 Training Algorithms

We implement both PPO and APG within a single unified training script, selected via `--algorithm {ppo,apg}`. Both share identical agent architectures, observation normalization, learning rate annealing, and Adam optimizer settings. The fundamental difference is how gradients are obtained. Table 1 summarizes the key distinctions.

Table 1: Comparison of PPO and APG algorithm components.

Component	**PPO**	**APG**
Environment interaction	`torch.no_grad()`	Computation graph preserved
Gradient source	Clipped surrogate + GAE	Backprop through dynamics
Value function	Required (advantage estimation)	Optional: bootstrap at segment boundaries (long-horizon only)
Action sampling	`Normal.sample()`	`rsample()` (reparameterization)
Rollout buffer	Stores $(o, a, \log p, r, V)$	No buffer needed
Update structure	Multiple epochs over stored batch	Multiple grad steps with fresh rollouts

The APG training loop follows a **stateful rollout** paradigm: the environment is reset once and its state is carried forward across gradient steps. Actions are sampled via the reparameterization trick,

$$\mathbf{a} = \boldsymbol{\mu}_{\boldsymbol{\theta}}(s) + \boldsymbol{\sigma}_{\boldsymbol{\theta}} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{10}$$

so that gradients flow through both $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ and $\boldsymbol{\sigma}_{\boldsymbol{\theta}} = \exp(\log \boldsymbol{\sigma}_{\boldsymbol{\theta}})$ into the environment dynamics. Each iteration anneals the learning rate, then performs N_{grad} gradient steps. Per step: a full episode, or segments of length L_{seg}, is rolled out with the computation graph intact; segment returns G_k are accumulated; a bootstrap value b_k estimates future return beyond the segment; a loss $\mathcal{L} = -\frac{1}{N}\sum_i \sum_k (G_k + b_k)$ is computed and backpropagated; and gradients are clipped before the optimizer step. Algorithm 1 gives the complete pseudocode.

Algorithm 1 APG Training Iteration

Require: Policy $\pi_{\boldsymbol{\theta}}$, environment E, learning rate η, horizon H, segment length L_{seg}

for grad_step $= 1, \ldots, N_{\text{grad}}$ **do**
  $\nabla_{\boldsymbol{\theta}} \leftarrow 0$
  policy_loss $\leftarrow 0$
  obs $\leftarrow E$.current_state
  for segment $k = 0, \ldots, \lceil H/L_{\text{seg}} \rceil - 1$ **do**
    $G_k \leftarrow 0$
    for $t = 0, \ldots, L_{\text{seg}} - 1$ **do**
      $a_t \leftarrow$ differentiable_action$(\pi_{\boldsymbol{\theta}}, \text{obs})$ ▷ reparameterization trick
      $\text{obs}', r_t \leftarrow E.\text{step}(a_t)$ ▷ graph preserved
      $G_k \leftarrow G_k + \gamma^t \cdot r_t$
      obs $\leftarrow$ obs$'$
    end for
    obs $\leftarrow$ obs.detach() ▷ break gradient chain
    $b_k \leftarrow$ bootstrap(k) ▷ MC or $V(s)$
    policy_loss $\leftarrow$ policy_loss $-$ mean$(G_k + b_k)$
  end for
  loss $\leftarrow$ policy_loss $+ c_v \cdot$ critic_loss ▷ optional critic
  loss.backward()
  clip_grad_norm_$(\boldsymbol{\theta}, \text{max_norm})$
  optimizer.step()
end for

4.3 Segmented Backpropagation

For environments with long horizons, backpropagating through the full episode can lead to vanishing/exploding gradients. APG supports **segmentation**, where the horizon H is divided into $K = \lceil H/L_{\text{seg}} \rceil$ segments and the gradient chain is broken at segment boundaries:

- Within each segment, gradients flow through the full dynamics chain.
- At segment boundaries, observations and environment state are detached.
- Future returns beyond each segment are estimated via a **bootstrap value**.

The implementation supports two bootstrap modes:

1. **MC bootstrap**: Pre-computes future returns by backward accumulation of observed rewards from later segments. These rewards are detached (not differentiable) but provide a fixed target for the current segment's policy loss:

$$G_k = \sum_{t=0}^{L_k-1} \gamma^t r_{k,t}, \qquad b_k = \gamma^{L_k}(G_{k+1} + b_{k+1}).$$

2. **Critic bootstrap**: Uses a learned value function $V_\phi(s)$ to estimate future returns at segment boundaries, properly discounted as in the presentation:

$$b_k = \gamma^{L_k} V_\phi(s_{k,\text{end}}).$$

   The critic is trained concurrently via MSE loss against segment returns plus bootstrap values, using a separate (or shared) optimizer learning rate.

4.4 Implementation Details

To realize the segmented backpropagation strategy introduced earlier and to ensure unimpeded gradient flow across diverse environments, this section focuses on four key engineering aspects of our APG implementation: differentiable dynamics construction under native PyTorch autograd, a gradient bridge for external physics engines, observation normalization, and network architecture details.

Differentiable dynamics via native PyTorch autograd. For PointMassSimple, PointMassNavigate and PushT, all of which are implemented entirely in PyTorch, APG differentiability is achieved without any special gradient bridge. The `*APGEnv` subclasses override `step()` to omit `torch.no_grad()` and replace all in-place state updates with out-of-place assignments. Concretely, at each step the action is scaled to a force/torque, the damping-then-force Euler update is expressed as

$$\mathbf{v}_{t+1} = \mathbf{v}_t \cdot (1 - c_d \Delta t) + \tfrac{\mathbf{f}}{m} \Delta t, \qquad \mathbf{p}_{t+1} = \text{clamp}(\mathbf{p}_t + \mathbf{v}_{t+1} \Delta t),$$

where every multiplication and addition is a new tensor with a live `grad_fn`. The reward is then computed from these live tensors, so a single `loss.backward()` propagates gradients through the entire chain $\mathbf{a} \to (\mathbf{f}, \tau) \to (\mathbf{v}, \omega) \to (\mathbf{p}, \theta) \to r$. The observation returned to the policy at each step is constructed from the live `self.pos` and `self.vel` tensors (which carry `grad_fn`) rather than from detached copies. This is essential for critic bootstrapping in the segmented setting: the value $V_\phi(s_{k,\text{end}})$ computed at the end of segment k can then propagate gradients back through the terminal observation to the policy parameters, effectively extending the gradient horizon past the segment boundary. Only `last_action` is stored detached, as it serves as a constant penalty baseline for the action-rate reward term. At segment boundaries, `detach_state()` replaces every state tensor with its detached clone, preventing gradient chains from accumulating across segments and causing "backward through the graph a second time" errors.

Gradient bridge for physics simulation. For FrankaReach, which uses the Newton Physics engine for GPU-accelerated forward kinematics, gradients cannot flow natively through PyTorch. We implement a custom `torch.autograd.Function` (`_NewtonStepFunc`) that bridges Warp's tape-based autodiff with PyTorch's autograd. In the forward pass, action tensors are converted to Warp arrays, Newton's forward kinematics are executed within a `wp.Tape` context, and the reward and end-effector pose are returned as *detached* PyTorch tensors (the EEF pose is not connected to the autograd graph). In the backward pass, the Warp tape is replayed in reverse to compute $\partial r/\partial \mathbf{a}$, which is returned to PyTorch's autograd graph. The $\partial \mathbf{p}_{\text{eef}}/\partial \mathbf{a}$ gradient is not propagated (EEF pose was detached in the forward pass); instead, differentiable joint-position observations are constructed in PyTorch to provide credit assignment through the joint-state component of the observation. This bridge enables APG through articulated-body kinematics without a native differentiable physics backend.

Observation normalization. Both algorithms use online observation normalization via a Welford-style running mean and standard deviation tracker (`RunningObsNormalizer`). Statistics are updated with detached observations after each rollout/gradient step, ensuring normalization does not interfere with APG gradient flow.

Network architecture. Both PPO and APG share a 2-layer MLP actor (LayerNorm is supported but disabled by default):

$$\text{obs\_dim} \to \text{Linear}(64) \to \text{Tanh} \to \text{Linear}(64) \to \text{Tanh} \to \text{Linear}(\text{action\_dim})$$

plus a learnable $\log \sigma$ parameter (initialized to -2.0, $\sigma \approx 0.135$). The critic (required for PPO; optional for APG in critic-bootstrap mode) uses a shallower MLP:

$$\text{obs\_dim} \to \text{Linear}(64) \to \text{Tanh} \to \text{Linear}(64) \to \text{Tanh} \to \text{Linear}(1)$$

All linear layers use orthogonal initialization with std $= \sqrt{2}$ (hidden), 0.01 (actor output), or 1.0 (critic output).

5 Environments

We evaluate on four continuous-control environments of increasing complexity, each implemented with both a black-box mode (PPO) and a differentiable mode (APG).

5.1 PointMassSimple

Task: Control a one-dimensional point mass and move it to a randomly sampled target position. This environment is the simplest benchmark in our suite: it removes obstacles and orientation while retaining continuous dynamics, stochastic initial states, and a differentiable reward.

State space (3-dimensional): See Table 2.

Table 2: PointMassSimple observation space.

Component	Dimensions	Description
Position	1	Point-mass position $p \in [-1, 1]$ at reset
Velocity	1	Scalar velocity v
Target position	1	Goal position $p_g \in [-1, 1]$ at reset

Action space: $\mathcal{A} = [-1, 1]$ (scalar force command).

Reset distribution: Position and target are sampled uniformly from $[-1, 1]$, while velocity is sampled from a small interval around zero. This gives both algorithms the same randomized start–goal distribution.

Dynamics (damped Euler integration, $\Delta t = 0.1$):

$$f_t \leftarrow \text{clip}(a_t, -1, 1) \tag{11}$$

$$v_{t+1} \leftarrow v_t + (f_t - 0.5v_t)\Delta t \tag{12}$$

$$p_{t+1} \leftarrow p_t + v_{t+1}\Delta t. \tag{13}$$

The implementation is written entirely in PyTorch, so gradients flow through $a_t \to f_t \to v_{t+1} \to p_{t+1}$ in APG mode.

Reward:

$$r_t = -\left(p_{t+1} - p_g\right)^2. \tag{14}$$

This dense quadratic reward supplies a smooth analytic signal throughout the state space while preserving a clear target-reaching objective.

Success condition: $|p - p_g| < 0.05$.

Episode length: 200 steps in the presentation experiments; in the unified training script the horizon can be overridden through `--max_episode_steps`.

APG gradient path: $a_t \to f_t \to v_{t+1} \to p_{t+1} \to r_t$.

5.2 PointMassNavigate

Task: Navigate a point mass from a random start to a random goal position while avoiding two randomly-placed circular obstacles.

State space (14-dimensional): See Table 3.

Table 3: PointMassNavigate observation space.

Component	Dimensions	Description
Position	2	$(x, y) \in [-1, 1]^2$
Velocity	2	(v_x, v_y)
Goal position	2	Target (x_g, y_g)
Last action	2	Previous force command
Obstacle 1	3	Position (x_1, y_1) + radius r_1
Obstacle 2	3	Position (x_2, y_2) + radius r_2

Action space: $\mathcal{A} = [-1, 1]^2$ (2D force vector, scaled by 5.0 N).

Dynamics (semi-implicit Euler, $\Delta t = 1/60\,\text{s}$):

$$\mathbf{v} \leftarrow \mathbf{v} \cdot (1 - c_d \Delta t) \tag{15}$$

$$\mathbf{v} \leftarrow \mathbf{v} + \frac{\mathbf{f}}{m} \Delta t \tag{16}$$

$$\mathbf{p} \leftarrow \text{clamp}\big(\mathbf{p} + \mathbf{v}\Delta t,\ [-1, 1]^2\big) \tag{17}$$

where $m = 1.0\,\text{kg}$, $c_d = 5.0$, $\mathbf{f} = \mathbf{a} \times 5.0\,\text{N}$.

Reward:

$$r = -\|\mathbf{p} - \mathbf{p}_g\| + 0.5 \exp\left(-\frac{\|\mathbf{p} - \mathbf{p}_g\|^2}{2 \cdot 0.05^2}\right) - 0.01\|\mathbf{v}\|^2 - 0.001\|\mathbf{a} - \mathbf{a}_{\text{prev}}\|^2 - 2.0 \sum_{i=1}^{2} \max(0, r_i - d_i)^2 \tag{18}$$

Success condition: $\|\mathbf{p} - \mathbf{p}_g\| < 0.03\,\text{m}$.

Episode length: 100 steps (default).

APG gradient path: $\mathbf{a} \to \mathbf{f} \to \mathbf{v} \to \mathbf{p} \to r$ (fully differentiable Euler integration).

5.3 PushT

Task: Push a T-shaped rigid block from a random initial pose to a random goal pose using external forces and torques.

State space (9-dimensional): See Table 4.

Table 4: PushT observation space.

Component	Dimensions	Description
Block position	2	$(x, y) \in [-0.25, 0.25]^2$
Block angle	1	$\theta \in [-\pi, \pi]$
Goal position	2	Target (x_g, y_g)
Goal angle	1	Target θ_g
Block velocity	2	(v_x, v_y)
Block angular velocity	1	ω

Action space: $\mathcal{A} = [-1, 1]^3$ (force x, force y, torque z).

T-block geometry: Top bar: $0.12 \times 0.03\,\text{m}$. Stem: $0.03 \times 0.09\,\text{m}$. Total height $\approx 0.12\,\text{m}$.

Dynamics (2D rigid body Euler, $\Delta t = 1/60\,\text{s}$):

$$\mathbf{v} \leftarrow \mathbf{v} \cdot (1 - c_d \Delta t), \qquad \omega \leftarrow \omega \cdot (1 - c_\omega \Delta t) \tag{19}$$

$$\mathbf{v} \leftarrow \mathbf{v} + \frac{\mathbf{f}}{m} \Delta t, \qquad \omega \leftarrow \omega + \frac{\tau}{I} \Delta t \tag{20}$$

$$\mathbf{p} \leftarrow \text{clamp}\big(\mathbf{p} + \mathbf{v}\Delta t,\ [-0.25, 0.25]^2\big)\,, \qquad \theta \leftarrow \text{wrap}(\theta + \omega \Delta t) \tag{21}$$

where $m = 0.1\,\text{kg}$, $I = 1.95 \times 10^{-4}\,\text{kg}{\cdot}\text{m}^2$, $c_d = 5.0$, $c_\omega = 3.0$, $\mathbf{f} = \mathbf{a}_{[0:2]} \times 5.0\,\text{N}$, $\tau = a_{[2]} \times 0.2\,\text{Nm}$.

Reward:

$$r = -\|\mathbf{p} - \mathbf{p}_g\| + 0.5 \exp\left(-\frac{\|\mathbf{p} - \mathbf{p}_g\|^2}{2 \cdot 0.02^2}\right) - 0.5\,|\text{wrap}(\theta - \theta_g)| - 0.001\,\|\mathbf{a} - \mathbf{a}_{\text{prev}}\|^2 \tag{22}$$

Success condition: $\|\mathbf{p} - \mathbf{p}_g\| < 0.01\,\text{m}$ and $|\text{wrap}(\theta - \theta_g)| < 0.1\,\text{rad}$.

Episode length: 100 steps (default).

APG gradient path: $\mathbf{a} \to (\mathbf{f}, \tau) \to (\mathbf{v}, \omega) \to (\mathbf{p}, \theta) \to r$.

5.4 FrankaReach

Task: Control a 7-DOF Franka FR3 manipulator to reach a target 6D end-effector pose (position + orientation).

State space (28-dimensional): See Table 5.

Table 5: FrankaReach observation space.

Component	Dimensions	Description
Joint positions	7	$q_1, \ldots, q_7$ (arm joints)
EEF position	3	(x, y, z) of end-effector
EEF quaternion	4	(x, y, z, w) orientation
Target position	3	Goal (x_g, y_g, z_g)
Target quaternion	4	Goal orientation (x_g, y_g, z_g, w_g)
Last action	7	Previous joint velocity command

Action space: $\mathcal{A} = [-1, 1]^7$ (delta joint positions, scaled by 0.2 rad).

Dynamics: Newton Physics engine with forward kinematics. Joint targets are computed as $q_{\text{target}} = \text{clamp}(q_{\text{current}} + \mathbf{a} \times 0.2,\ q_{\min}, q_{\max})$.

Target sampling: Positions sampled uniformly in $x \in [0.05, 0.70]$, $y \in [-0.45, 0.45]$, $z \in [0.20, 0.95]$ m. Orientations sampled with tilt up to $\pm 60°$ from the default downward pose.

Reward:

$$r = -0.2\,\|\mathbf{p}_{\text{eef}} - \mathbf{p}_{\text{target}}\| + 0.1 \exp\left(-\frac{\|\mathbf{p}_{\text{eef}} - \mathbf{p}_{\text{target}}\|^2}{2 \cdot 0.1^2}\right) - 0.1 \cdot d^2_{\text{quat}}(\mathbf{q}_{\text{eef}}, \mathbf{q}_{\text{target}}) - 0.0001\,\|\mathbf{a} - \mathbf{a}_{\text{prev}}\|^2 \tag{23}$$

Success condition: $\|\mathbf{p}_{\text{eef}} - \mathbf{p}_{\text{target}}\| < 0.005\,\text{m}$ and $d^2_{\text{quat}}(\mathbf{q}_{\text{eef}}, \mathbf{q}_{\text{target}}) < 0.1$, where $d^2_{\text{quat}}(\mathbf{q}_1, \mathbf{q}_2) = \min(\|\mathbf{q}_1 - \mathbf{q}_2\|^2,\ \|\mathbf{q}_1 + \mathbf{q}_2\|^2)$ handles the quaternion double cover.

Episode length: 30 steps (default).

APG gradient path: $\mathbf{a} \to q_{\text{target}} = \text{clamp}(q + \mathbf{a} \times 0.2) \to \text{FK}(q_{\text{target}}) \to r$ (reward differentiable via Warp tape); $\mathbf{a} \to q_{\text{target}}$ also constructed in PyTorch for differentiable joint-position observations.

The gradient computation uses a custom `torch.autograd.Function` that bridges the Warp-tape autodiff in Newton Physics with PyTorch's autograd system (see Section 4.4).

5.5 Environment Complexity Comparison

Table 6 compares the four environments across key properties.

Table 6: Comparison of environment properties.

Property	**PointMassSimple**	**PointMass**	**PushT**	**FrankaReach**
Observation dim	3	14	9	28
Action dim	1	2	3	7
Dynamics	1D point mass (Euler)	2D Point Mass (Euler)	2D rigid body (Euler)	Articulated body (FK)
DOF	1 (pos)	2 (pos)	3 (pos + rot)	7 (joints)
Max episode steps	200	100	100	30
Gradient bridge	Native PyTorch	Native PyTorch	Native PyTorch	Warp ↔ PyTorch
Obstacles/constraints	None	2 circles	Workspace bounds	Joint limits
Orientation	No	No	Yes (θ)	Yes (quaternion)

6 Experiments

6.1 Experimental Setup

Shared hyperparameters (identical for PPO and APG unless noted): See Table 7.

Table 7: Shared hyperparameters.

Hyperparameter	**Value**
Learning rate	2.5×10^{-4}
LR schedule	Linear annealing to 0
Discount factor γ	0.99
Max gradient norm	0.5
Optimizer	Adam ($\epsilon = 10^{-5}$)
Observation normalization	Welford running mean/std
Number of parallel envs	4
Network architecture	2-layer MLP (64 units, Tanh, no LayerNorm)
Total environment steps	500,000

PPO-specific hyperparameters: See Table 8.

APG-specific hyperparameters: See Table 9.

6.2 Evaluation Protocol

We employ a rigorous evaluation protocol to ensure fair comparison:

Table 8: PPO-specific hyperparameters.

Hyperparameter	**Value**
Steps per rollout	128
Minibatches	4
Update epochs	4
GAE λ	0.95
Clip coefficient ϵ	0.2
Value function coefficient	0.5
Entropy coefficient	0.01
Clipped value loss	Yes
Advantage normalization	Yes

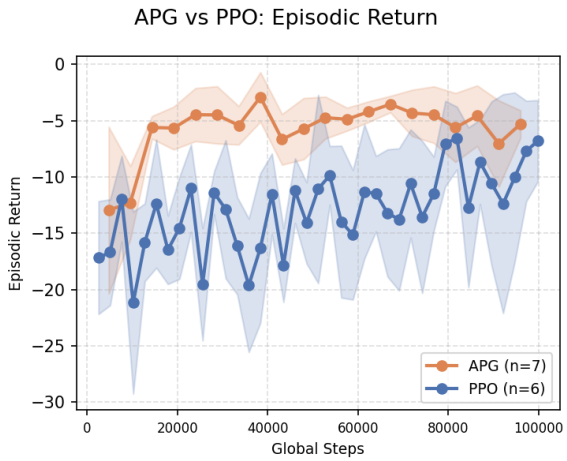
Table 9: APG-specific hyperparameters.

Hyperparameter	**Value**
Gradient steps per iteration	8
Segment length	Full episode
Bootstrap mode	Critic

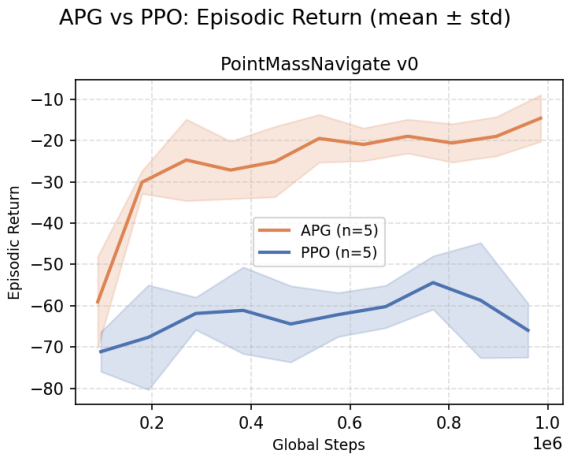
1. **Deterministic evaluation:** At regular intervals, we run a fixed number of evaluation episodes in a separate black-box environment using the greedy policy: argmax over logits for discrete action spaces, or the clamped actor mean $\text{clip}(\boldsymbol{\mu}_{\boldsymbol{\theta}}(s), -1, 1)$ for continuous spaces. Observations are normalized via the shared running normalizer (no update during eval). The following metrics are collected per evaluation checkpoint:
   - **Episodic return**: mean undiscounted return over completed episodes.
   - **Episodic length**: mean number of steps per episode.
   - **Success rate**: fraction of episodes that terminate by reaching the goal, as opposed to timeout truncation.
   - **Mean time to success**: mean episode length conditioned on successful episodes only.
2. **Multi-axis logging:** Metrics are recorded against two independent axes to enable fair comparison between PPO and APG:
   - **Environment steps**: Total number of environment interactions (measures sample efficiency).
   - **Gradient steps**: Total number of optimizer updates (measures compute efficiency, equalized across algorithms for controlled comparison).
3. **Seed sweep:** Results are reported as mean $\pm$ std over multiple random seeds.

6.3 Experiment Results

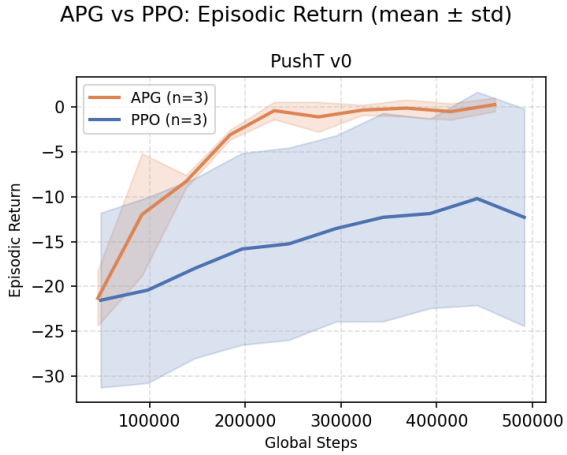
Figure 1 shows the episodic return curves for all four environments, plotted against total gradient steps to enable a fair compute-controlled comparison between PPO and APG. Table 10 summarises the final returns.



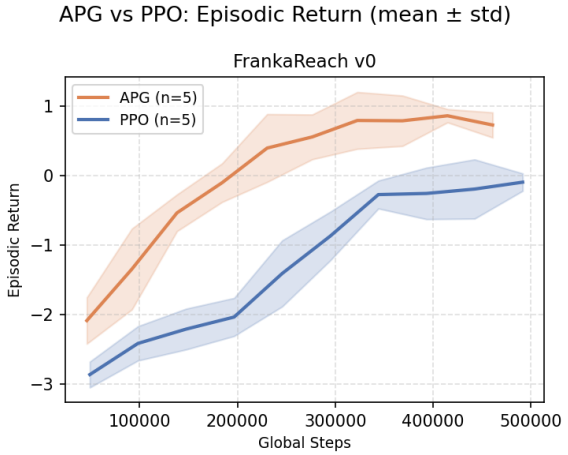
(a) PointMassSimple



(b) PointMassNavigate



(c) PushT



(d) FrankaReach

Figure 1: Episodic return (mean ± std) vs. total gradient steps for PPO (blue) and APG (orange). The x-axis is equalized: both algorithms execute the same total number of gradient updates.

Table 10: Final performance (episodic return). Results reported as mean ± std over multiple seeds.

Environment	**PPO (mean ± std)**	**APG (mean ± std)**
PointMassSimple	-7.14 ± 3.87	-5.66 ± 1.28
PointMassNavigate	-64.89 ± 11.92	-14.59 ± 3.36
PushT	-12.29 ± 12.13	0.29 ± 0.76
FrankaReach	-0.09 ± 0.13	0.73 ± 0.08

Table 11: Sample efficiency: environment steps to reach performance threshold.

Environment	**Success Ratio**	**PPO Steps**	**APG Steps**	**APG Reduction**
PointMassSimple	100%	69120	38400	1.80x
PointMassNavigate	20%	263360	192000	1.37x
PushT	100%	147456	138240	1.07x
FrankaReach	20%	1000000	230400	4.34x

Table 12: Learning efficiency: gradient steps to reach performance threshold.

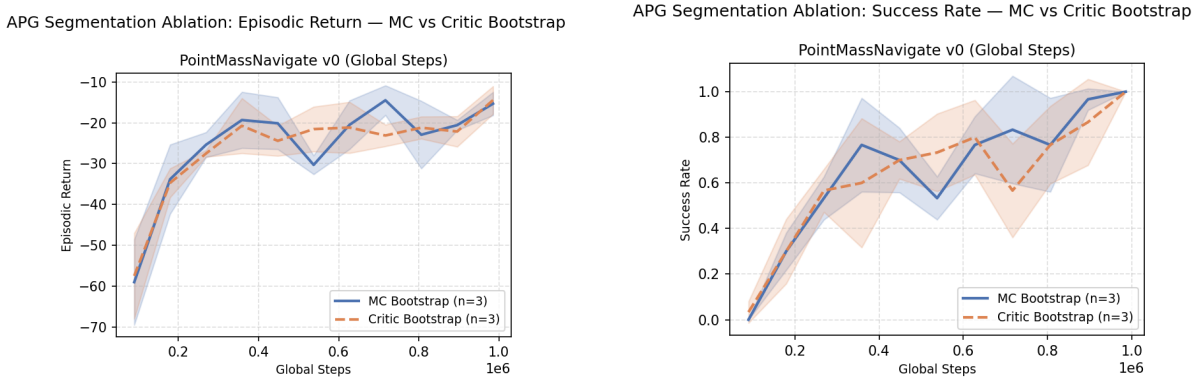
Environment	Success Ratio	PPO Grad Steps	APG Grad Steps	APG Speedup
PointMassSimple	100%	2150	320	6.72x
PointMassNavigate	20%	1104	120	9.2x
PushT	100%	560	144	3.89x
FrankaReach	20%	3824	240	15.93x

6.4 Ablation: Bootstrap Mode for Segmented APG

We conduct an ablation study on PointMassNavigate to isolate the effect of the bootstrap strategy in segmented APG (segment length $L = 25$, horizon $T = 100$, giving 4 segments per episode). Two modes are compared across 3 seeds each:

- **MC bootstrap**: future returns for each segment are pre-computed by backward accumulation of detached rewards from all subsequent segments, properly discounted by γ^{L_k} at each segment boundary.
- **Critic bootstrap**: a learned value function $V_\phi(s)$ estimates the discounted future return at each segment boundary ($b_k = \gamma^{L_k} V_\phi(s_{k,\text{end}})$). The critic is trained concurrently via MSE regression to segment-level bootstrapped returns. Crucially, $s_{k,\text{end}}$ is kept in the computation graph (not detached), so $\nabla_{\boldsymbol{\theta}} b_k$ extends the effective gradient horizon through the critic to the terminal state of each segment.

Both modes reach a success rate of 1.0 by the end of training. The critic bootstrap converges to a slightly higher final episodic return (-14.44 ± 3.38) than MC bootstrap (-15.31 ± 2.84), though the difference is within one standard deviation given three seeds. The critic bootstrap exhibits marginally faster early-phase improvement, consistent with the expected benefit of a learned value function reducing variance in the return estimate at segment boundaries.



(a) Return vs. environment steps

(b) Success rate vs. environment steps

Figure 2: Bootstrap-mode ablation on PointMassNavigate.

6.5 Ablation: Segment Length with MC and Critic Bootstrap

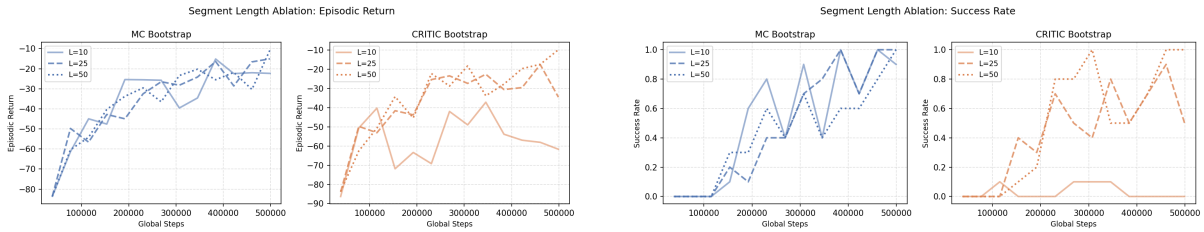
To understand how segment length L interacts with the bootstrap strategy, we sweep $L \in \{10, 25, 50\}$ under both MC and critic bootstrap on PointMassNavigate (horizon $T = 100$, one seed per condition). This isolates the credit-assignment rangeshorter segments truncate gradient chains more aggressively, relying more heavily on the bootstrap estimate to carry future-return information across boundaries.

Table 13 summarizes the results.

MC bootstrap is robust across all three segment lengths. Even at $L = 10$ (10 segments per episode) it achieves a success rate of 0.9 and converges to a reasonable final return of -22.3. Performance improves monotonically with L: at $L = 25$ and $L = 50$ the agent reaches a success rate of 1.0, with final returns of -15.1 and -10.5 respectively. The consistent gradient signal from pre-computed discounted returns makes MC bootstrap tolerant of shorter horizons.

Critic bootstrap exhibits a strong sensitivity to L. At $L = 10$, the critic fails to provide reliable bootstrap targets early in training, and the agent converges to a degenerate policy with zero success rate and a return of -61.7. At $L = 25$ the critic partially recovers (success rate 0.5, return -34.5), and at $L = 50$ it matches MC bootstrap (-9.8, success 1.0). The critic's value estimates require a sufficient gradient horizon to stabilize: when segments are too short, the critic's bootstrapped targets are too noisy to extend effective credit assignment beyond the segment boundary, and the additional critic loss introduces conflicting gradients.

The results suggest that MC bootstrap should be preferred as the default for short-to-medium segment lengths, while critic bootstrap becomes competitive only when L is large enough to supply stable training signal to the value network.



(a) Final return

(b) Success rate

Figure 3: Segment-length ablation on PointMassNavigate.

Table 13: Segment length ablation: final episodic return and success rate on PointMassNavigate (1 seed per condition, 500K steps).

Bootstrap	L	**Final Return**	**Success Rate**
MC	10	-22.3	0.90
MC	25	-15.1	1.00
MC	50	-10.5	1.00
Critic	10	-61.7	0.00
Critic	25	-34.5	0.50
Critic	50	-9.8	1.00

7 Discussion and Conclusion

7.1 Summary

We presented a unified framework for comparing Analytic Policy Gradients (APG) and Proximal Policy Optimization (PPO) across four differentiable continuous control environments. Experiments on PointMassSimple, PointMassNavigate, PushT, and FrankaReach yield four concrete findings:

1. **Gradient quality vs. variance:** APG computes exact policy gradients by backpropagating through dynamics, bypassing the variance inherent in Monte Carlo advantage

estimation. Across the tested environments APG converges to high success rates and higher final returns, confirming that exact gradients provide a reliable training signal.

2. **Sample efficiency:** APG requires substantially fewer gradient steps than PPO to reach comparable performance thresholds (Table 12): 6.7× fewer on PointMassSimple, 9.2× fewer on PointMassNavigate, 3.9× on PushT, and 15.9× on FrankaReach, demonstrating that the per-step quality of analytic gradients more than compensates for PPO's multi-epoch update schedule.

3. **Segmentation and bootstrap strategy matter:** The segment length ablation (Section 6.5) shows that MC bootstrap degrades gracefully as L shrinks (success rate 0.9 at $L = 10$), while critic bootstrap collapses entirely at $L = 10$ (success rate 0.0) and recovers only at $L = 50$. MC bootstrap is therefore the more robust default for short-to-medium horizons; critic bootstrap offers competitive final performance only when the segment length is large enough to stabilize its value targets.

4. **Gradient bridge feasibility:** Our custom `torch.autograd.Function` demonstrates that analytic policy gradients are practical for GPU-accelerated physics engines that do not natively expose PyTorch-compatible derivatives, broadening APG beyond analytically specified environments.

7.2 Limitations

1. **Environment differentiability requirement:** APG fundamentally requires differentiable environment dynamics. This limits applicability to simulations and excludes real-world training or environments with non-differentiable components (contacts, collisions with discrete events).

2. **Gradient chain length:** Long episodes can lead to vanishing or exploding gradients despite segmentation. The effectiveness of APG may be sensitive to the choice of segment length and bootstrap strategy.

3. **Compute overhead:** Maintaining the computation graph through environment rollouts incurs memory and compute overhead compared to PPO's detached rollouts, particularly for complex environments like FrankaReach.

7.3 Future Work

1. **Hybrid APG–PPO algorithms with trust-region constraints.** The complementary strengths of APG and PPO suggest a natural synthesis: APG supplies low-variance analytic gradients, while PPO's clipped surrogate objective provides a conservative trust-region constraint that prevents catastrophic policy updates. Prior work on short-horizon actor-critic (SHAC) methods [4] and differentiable simulation-based actors [5] demonstrates that combining analytic rollouts with a learned value baseline can substantially stabilize training. A principled hybrid could apply the PPO ratio-clipping objective directly to the analytic gradient update, bounding effective policy step sizes while preserving gradient quality. Our unified implementation, which shares actor-critic architecture and observation normalization across both algorithms, is well-positioned to conduct a controlled investigation of this design space.

2. **APG through learned differentiable world models.** The differentiability requirement currently restricts APG to analytically specified simulators. A natural generalization is to treat learned neural world models as the differentiable substrate, backpropagating policy gradients through model-predicted rollouts rather than ground-truth dynamics.

Recent model-based RL systems such as DreamerV3 [6] and TD-MPC2 [7] demonstrate that latent imagination with differentiable recurrent transitions enables competitive policy optimization without environment rollouts. However, gradient compounding through approximate models introduces model-bias error that can dominate the variance reduction benefit. A systematic study comparing APG gradient quality under ground-truth versus learned dynamics—as a function of model capacity, rollout horizon, and Lipschitz constant—would clarify when learned-model gradients remain reliable signals and when model errors corrupt the optimization landscape.

3. **Principled adaptive segment-length scheduling.** Our segment length ablation (Section 6.5, Table 13) reveals a clear bias–variance trade-off: shorter segments reduce memory overhead and gradient chain length but truncate credit assignment, and the impact is particularly severe for critic bootstrap. A theoretically principled curriculum could adapt L_{seg} online based on the gradient signal-to-noise ratio [8] or the convergence of the critic's bootstrap targets. The optimal segment length balances the bias introduced by truncated BPTT against gradient variance, an objective analogous to the λ selection problem in GAE. Adaptive schedules grounded in this bias–variance decomposition, potentially implemented via meta-gradient methods that differentiate through the segment-length choice, would provide a more robust alternative to the fixed-length ablation studied here.

References

References

[1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

[2] Schulman, J., Moritz, P., Levine, S., Abbeel, P., & Jordan, M. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. *ICLR 2016*.

[3] Werbos, P. J. (1990). Backpropagation Through Time: What It Does and How to Do It. *Proceedings of the IEEE*, 78(10), 1550–1560.

[4] Guo, X., Xu, Z., Liu, C., & Wen, Y. (2023). Short-Horizon Actor-Critic. *arXiv preprint arXiv:2304.14457*.

[5] Xu, Z., Guo, X., Liu, C., & Wen, Y. (2022). Accelerated Policy Learning with Parallel Differentiable Simulation. *ICLR 2023*.

[6] Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2023). Mastering Diverse Domains through World Models. *arXiv preprint arXiv:2301.04104*.

[7] Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, Robust World Models for Continuous Control. *ICLR 2024*.

[8] Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the Difficulty of Training Recurrent Neural Networks. *ICML 2013*, 1310–1318.

[9] Degrave, J., Hermans, M., Dambre, J., & Wyffels, F. (2019). A Differentiable Physics Engine for Deep Learning in Robotics. *Frontiers in Neurorobotics*, 13, 6.

[10] Zhong, Y., Han, J., & Berseth, G. (2022). Differentiable Simulation for Physical System Identification. *IEEE Robotics and Automation Letters*, 7(1), 153–160.

[11] Suh, H. J., Simchowitz, M., Zhang, K., & Tedrake, R. (2022). Do Differentiable Simulators Give Better Policy Gradients? *ICML 2022*.

[12] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ICML 2018*.

[13] Amari, S. (1998). Natural Gradient Works Efficiently in Learning. *Neural Computation*, 10(2), 251–276.

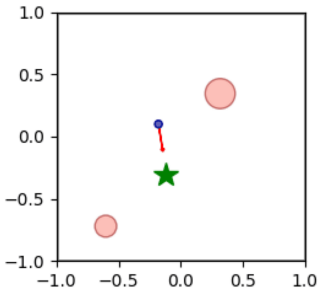
[14] Martens, J., & Grosse, R. (2015). Optimizing Neural Networks with Kronecker-Factored Approximate Curvature. *ICML 2015*.

[15] Williams, R. J. (1992). Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3–4), 229–256.

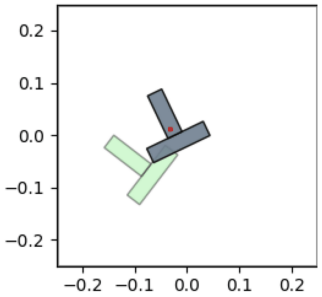
[16] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust Region Policy Optimization. *ICML 2015*, 1889–1897.

A Environment Visualizations

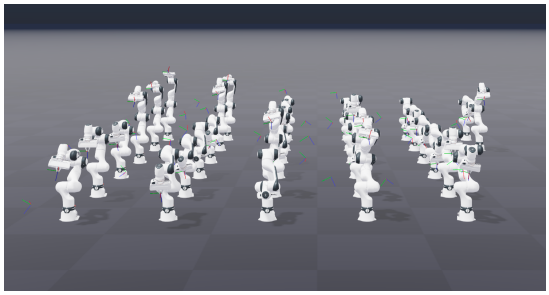
Rendered frames and GIFs for the custom differentiable environments are produced by the environment-specific renderers during evaluation. PointMassSimple renders the one-dimensional track, target marker, success band, current position, and velocity arrow; PointMassNavigate, PushT, and FrankaReach use analogous task-specific visualizations.



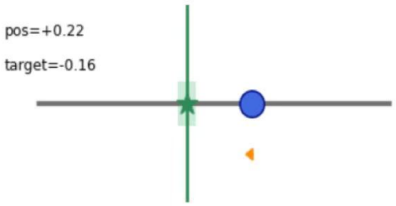
(a) PointMassNavigate



(b) PushT



(c) FrankaReach



(d) SimplePointMass

Figure 4: Environment renderings of the four continuous control tasks.